Countermeasures and Defence Mechanisms for Drone Swarm Threats

Major Project 1 (CS499) — Mid-Semester Evaluation

Group III

Mrutyunjay Kumar Rao (22CSE1019)
Nischay Khobragade (22CSE1023)

Guide: Dr. Pravati Swain

March 2026

National Institute of Technology Goa
Department of Computer Science and Engineering

Overview & Structure

1. **Phase 1 Recap & Limitations**
   Where Phase 1 ended and why it was not enough.
2. **Communication Graph & Fiedler Value**
   Modeling the swarm as a graph; algebraic connectivity λ_2.
3. **System Pipeline & DBSCAN Deployment**
   Complete 4-phase architecture and intelligent jammer initialization.
4. **Agent Design — Observation, Actor & Critic**
   5D state vector, neural network architectures, and CTDE.
5. **Reward Function & TD Learning**
   Multi-objective reward and temporal difference error.

I: Phase 1 Recap & Limitations

Where Phase 1 Ended — and Why It Was Not Enough

What Phase 1 Achieved

Component	Phase 1 Approach
Algorithm	Q-Learning + Tile Coding
Reward	Max received jamming power
Scenarios	2v3, 4v3, 6v3

Critical Limitations

- Wrong objective — maximizing power at individual drones does not measure swarm coordination loss.
- No graph metric — no way to know if the swarm is still connected as a network.
- Q-table cannot scale — state space explodes beyond 6 agents.
- Single frequency — only 2.4 GHz; real swarms use multiple bands.

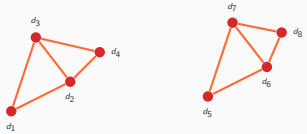
Core Gap

High jamming power at individual drones $\neq$ swarm fragmentation

Phase 2 objective: prove and optimize the exact mathematical condition for swarm disconnection.

II: Communication Graph & Algebraic Connectivity

Enemy Swarm $\rightarrow$ Communication Graph $G = (V, E)$



$V = \{d_1, d_2, \ldots, d_n\}$ — drones are nodes
$E = \{(i, j)\}$ — active radio links are edges

Adjacency Matrix (5×5 subset):

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Edge Condition — FSPL Model

$$P_R(i, j) = P_{tx} \cdot \left(\frac{c}{4\pi f \cdot d_{ij}} \right)^2$$

Edge exists if:

- $P_R(i, j) \geq P_{\text{sens}} = -90$ dBm
- AND link is not actively jammed

$d_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|_2$

Graph Laplacian

$$L = D - A$$

- D = diagonal degree matrix
- $D[i, i]$ = number of links drone i has

Algebraic Connectivity λ_2 — One Number That Measures Swarm Coordination

Eigen decomposition of the Laplacian L

$$L \cdot \mathbf{v} = \lambda \cdot \mathbf{v} \qquad \text{Eigenvalues: } 0 = \lambda_1 \leq \lambda_2 \leq \lambda_3 \leq \cdots \leq \lambda_n$$

$\lambda_2 =$ **Fiedler Value** $=$ **Algebraic Connectivity**

Dense Graph

$\lambda_2 = 0.82$

Swarm fully coordinated

Dangerous

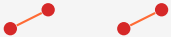
Sparse Graph

$\lambda_2 = 0.21$

Partially disrupted

Degraded

Fragmented



$\lambda_2 = 0.00$

Swarm fragmented

Mission Complete

Proposition 1 — Fiedler (1973): $\lambda_2(G) = 0 \iff G$ is disconnected ;
$\lambda_2(G) > 0 \iff G$ is connected

When our system drives λ_2 to zero, swarm fragmentation is provably guaranteed.

III: System Pipeline & DBSCAN Deployment

Complete System Pipeline — 4 Phases

PHASE 1 — Environment Setup
Spawn N enemy drones · Build graph G · Compute initial $\lambda_2(0)$

↓

PHASE 2 — Clustering & Deployment
Run DBSCAN on enemy positions · Identify k centroids · Deploy M jammers

↓

PHASE 3 — MARL-PPO Training Loop
5D observation → Actor outputs velocity + band → λ_2 recomputed → PPO update

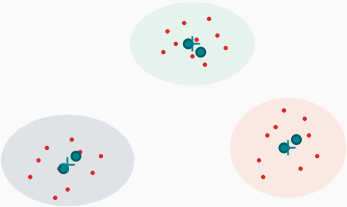
↓

PHASE 4 — Deployment
Load trained Actor weights · Deterministic inference · Monitor λ_2

System Parameters

N (enemies)	30
M (jammers)	6
Arena	$100\,\text{m} \times 100\,\text{m}$
Episode length	100 steps
Training	1500 episodes
Success	$\lambda_2 < 0.2 \cdot \lambda_2(0)$

DBSCAN — Intelligent Jammer Initialization



$\varepsilon = 30\,\mathrm{m}$ (neighborhood radius)
$\mathrm{minPts} = 2$ (minimum cluster size)
$k = 3$ clusters detected
$M = 6$ jammers $\rightarrow$ 2 per cluster

Why DBSCAN?

- Enemy swarms naturally form spatial clusters.
- Random jammer placement wastes the first 100–200 steps just repositioning.
- DBSCAN finds cluster structure from enemy (x, y) positions — **no labels needed**.
- Jammers start at centroids $\rightarrow$ immediately in effective range.

DBSCAN Algorithm — 3 Steps

1. For each drone, find all drones within $\varepsilon = 30\,\mathrm{m}$.
2. If $\geq$ minPts$= 2$ neighbors $\rightarrow$ core point $\rightarrow$ form cluster.
3. Compute centroid μ_k of each cluster k.

IV: Agent Design — Observation, Actor & Critic

What Each Jammer Sees — 5D State Observation

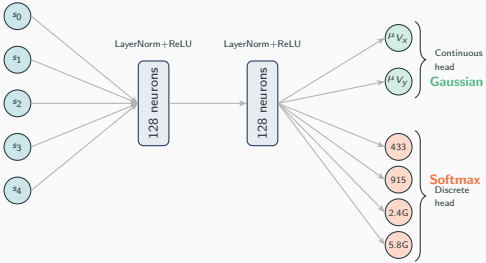
Index	Feature	Formula	Range	Meaning
$s[0]$	dist_to_centroid	$\lVert \mathbf{p}_j - \mu_k \rVert \,/\, \text{arena}$	$[0, 1]$	How far am I from my assigned cluster center?
$s[1]$	cluster_density	$n_k \,/\, N$	$[0, 1]$	What fraction of enemies are in my cluster?
$s[2]$	dist_to_others	$\overline{\lVert \mathbf{p}_j - \mathbf{p}_{j'} \rVert} \,/\, \text{arena}$	$[0, 1]$	How spread out am I from teammate jammers?
$s[3]$	coverage_overlap	pairs within $2R_{\text{jam}}$ / total	$[0, 1]$	Am I wasting coverage by overlapping a teammate?
$s[4]$	band_match	$\mathbf{1}[b_j = b_{\text{enemy}}]$	$\{0, 1\}$	Is my frequency matching the enemy band?

Normalization

All values clipped to [0, 1] before entering the neural network: $\mathbf{s}_{\text{norm}} = \text{clip}(\mathbf{s},\ 0,\ 1)$

Prevents any single feature from dominating the network activations

Actor Network — Mapping 5D Observation to Actions



Layer Sizes

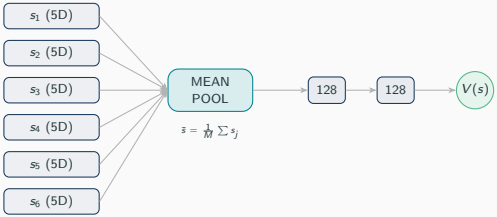
$5 \to 128$: each neuron learns a specific combination of the 5 input features.

$128 \to 128$: second layer combines simple patterns into complex jamming strategies.

Total: $(5 \times 128) + (128 \times 128) + (128 \times 6) \approx 18{,}000$ trainable weights.

Parameter sharing: all $M = 6$ jammers use ONE Actor network. Each jammer passes its own local 5D observation. Different inputs $\to$ different actions $\to$ coordinated behaviour.

Critic Network — Estimating Team Value $V(s)$



What Does $V(s)$ Mean?

$V(s) = 0.74$ means:
"From this team configuration, I expect 0.74 total reward over the remaining episode."

High $V(s) \rightarrow$ jammers well positioned, correct bands, λ_2 already falling.

Low $V(s) \rightarrow$ jammers misplaced, wrong frequency, λ_2 still high.

Why Mean Pooling?

Critic must see the whole team, but input size must stay fixed. Mean pool collapses $M \times 5 \rightarrow 5$ — scales to any number of jammers without retraining.

Centralized Training / Decentralized Execution (CTDE)

Training: Critic uses pooled global state $\rightarrow$ better gradient estimates for Actor.
Deployment: Critic is completely discarded $\rightarrow$ each jammer runs Actor alone using only its own 5D

V: Reward Function & TD Learning

Reward Function — 5 Terms Teaching the Agent What "Good" Means

Term	Formula	Weight	Dir.	Purpose
λ_2 Reduction	$1 - \lambda_2(t) \,/\, \lambda_2(0)$	$\omega_1 = 1.0$	$\uparrow$	**PRIMARY** — swarm fragmentation
Band Match	$\frac{1}{M} \sum \mathbf{1}[b_j = b_{\text{enemy}}]$	$\omega_2 = 0.3$	$\uparrow$	Correct frequency selection
Cluster Proximity	$\frac{1}{M} \sum \exp(-\kappa \cdot d_{\text{centroid}})$	$\omega_3 = 0.2$	$\uparrow$	Stay near assigned cluster
Energy Penalty	$\frac{1}{M} \sum \lVert \mathbf{v}_j \rVert^2 / v_{\max}^2$	$\omega_4 = 0.1$	$\downarrow$	Discourage unnecessary movement
Overlap Penalty	overlapping pairs / total pairs	$\omega_5 = 0.2$	$\downarrow$	Spread coverage across clusters

Total Reward

$$R(t) = \omega_1 \cdot (\lambda_2 \text{ reduction}) + \omega_2 \cdot (\text{band match}) + \omega_3 \cdot (\text{proximity}) - \omega_4 \cdot (\text{energy}) - \omega_5 \cdot (\text{overlap})$$

Temporal Difference Error — How the Agent Measures Surprise

Core Idea

The Critic predicts expected future reward at state s_t. After one step, we see what actually happened.

TD error = gap between prediction and reality.

Step 1 — Critic prediction at time t:

$V(s_t)$ = "From this state, I expect X total future reward"

Step 2 — Better estimate after one step:

$$G_t \approx r_t + \gamma \cdot V(s_{t+1})$$

r_t = actual reward; $\gamma = 0.99$

Step 3 — TD Error:

$$\boxed{\delta_t = r_t + \gamma \cdot V(s_{t+1}) - V(s_t)}$$

$\delta_t > 0$ — Better than expected

Action was good → increase its probability.

$\delta_t = 0$ — Matched prediction

Action was average → no change needed.

$\delta_t < 0$ —

Action was bad → decrease its probability.

δ_t is computed for **every timestep** in the buffer after each rollout. It is the raw signal from which Advantage and GAE are both derived. Without δ_t there is no learning signal — it is the engine of all RL.

From TD Error to GAE — Computing How Good Each Action Truly Was

Why Not Use δ_t Directly?

δ_t is only one step of information.
It fully trusts $V(s_{t+1})$, which is estimated by the Critic.

Early in training $\rightarrow$ Critic is wrong $\rightarrow$ δ_t is wrong $\rightarrow$ bad learning signal.

We need to **look further ahead** for a more accurate signal.

Advantage A_t

A_t = "How much better was THIS action compared to average?"

$A_t > 0 \rightarrow$ above average $\rightarrow$ reinforce
$A_t < 0 \rightarrow$ below average $\rightarrow$ suppress

True: $A_t = Q(s_t, a_t) - V(s_t)$
But Q is unknown $\rightarrow$ we must **estimate** it.

Generalised Advantage Estimation (GAE)

1-step: $\hat{A}_t^{(1)} = \delta_t$
2-step: $\hat{A}_t^{(2)} = \delta_t + \gamma\delta_{t+1}$
n-step: $\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}$

GAE — exponentially weighted sum of all n-step estimates:

$$\boxed{A_t^{\text{GAE}} = \delta_t + (\gamma\lambda)\, A_{t+1}^{\text{GAE}}}$$

Computed backwards from $t = T$ to $t = 0$.

VI: PPO — Proximal Policy Optimization

PPO — Why Standard Policy Gradient Breaks and How PPO Fixes It

Standard Policy Gradient — The Problem

$$\theta \leftarrow \theta + \alpha \cdot A_t \cdot \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

If A_t is large, the weight update is HUGE. Policy jumps catastrophically in a single update — destroying thousands of steps of learning.

No control over how much the policy changes per update.

PPO Key Insight — Probability Ratio

Measure how much the policy has changed:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

- $r_t = 1.0 \rightarrow$ policy unchanged
- $r_t = 1.5 \rightarrow$ 50% more likely
- $r_t = 0.6 \rightarrow$ 40% less likely

PPO keeps $r_t \in [1-\varepsilon,\ 1+\varepsilon] = [0.8,\ 1.2]$

No single update can change any action probability

Clipped Objective

$$L^{\text{CLIP}} = \mathbb{E}_t\Big[\min\Big(r_t A_t,\ \text{clip}(r_t,\ 1-\varepsilon,\ 1+\varepsilon)\, A_t\Big)\Big]$$

min() always picks the more conservative term.

Improvements capped at 20% $\rightarrow$ stable trust region.

Entropy Bonus

$H(\pi) = -\sum_a \pi(a|s) \log \pi(a|s)$

Prevents premature collapse to one action.

$L_{\text{entropy}} = -c_1 \cdot H(\pi_\theta), \quad c_1 = 0.01$

PPO Training — One Complete Update Cycle

Step 1 — Collect Experience: All 6 jammers run for 2048 steps using current policy. Store: state, action, reward, $\log \pi$, $V(s)$.

Step 2 — Compute GAE Advantages: Walk backwards through buffer: $A_t = \delta_t + \gamma\lambda \cdot A_{t+1}$. Every step now has an advantage estimate.

Step 3 — Normalize Advantages: $A_t \leftarrow (A_t - \mu)/\sigma$. All advantages on same scale $\rightarrow$ stable gradients.

Step 4 — 10 Epochs of Mini-batch Updates: Shuffle buffer $\rightarrow$ split into batches of 256. For each batch: compute r_t, clipped PPO loss, Critic MSE loss, backprop $\rightarrow$ update weights.

Step 5 — Gradient Clip + Adam: Clip gradient norm to 0.5. Adam adjusts each weight individually: $W_{\text{new}} = W_{\text{old}} - \text{lr} \cdot \text{gradient}$.

Step 6 — Reset and Repeat: $\pi_{\text{old}} \leftarrow \pi_{\text{new}}$. Clear buffer. Back to Step 1.

VII: Training Results & Evaluation

Training Results

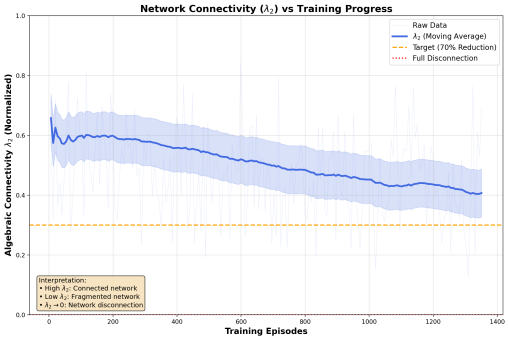


Figure 1: λ_2 vs. Training Episodes

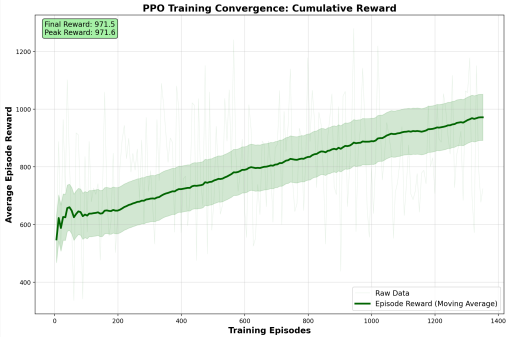
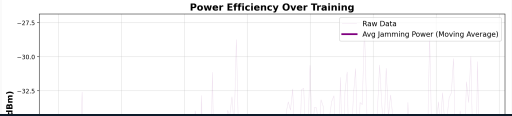
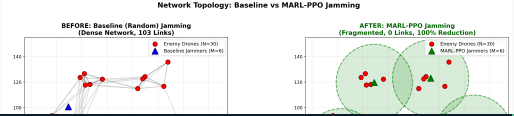


Figure 3: Reward vs. Training Episodes





Dataset Evaluation (Kaggle Benchmark)

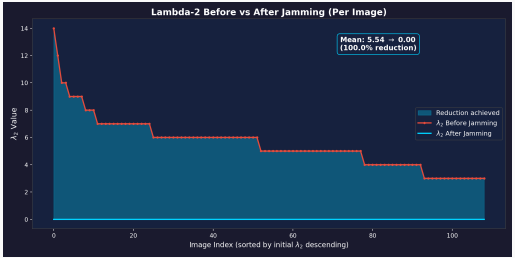


Figure 5: λ_2 Before vs. After — Kaggle Dataset

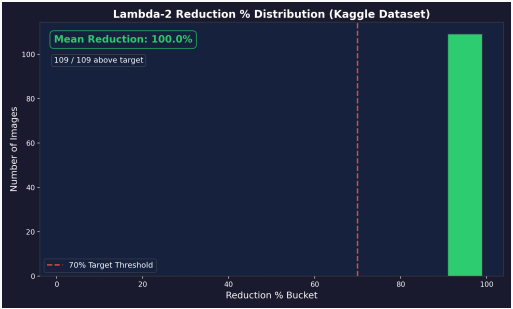


Figure 6: Reduction Distribution — Kaggle Dataset

Final Demonstration Video

Live demonstration of trained MARL agents fragmenting an enemy swarm.

Thank You

Questions & Discussion